

• 1. 결정 트리

학습목표



- 성능이 좋고 이해하기 쉬운 트리 알고리즘에 대해 배웁니다.
- 알고리즘의 성능을 최대화하기 위한 하이퍼파라미터 튜닝을 실습합니다.

레드와인과 화이트와인 분류하기



병와인을 캔와인으로 판매 : 특수캔으로 맛과 향을 유지하도록 제작

➔ 문제 : 레드/화이트 표시 누락

➔ 도수, 당도, pH 값으로 와인 종류 구별하는 방법

로지스틱 회귀로 와인 분류하기



- 로지스틱 회귀로 와인 분류하기
 - 6,497개의 와인 샘플 데이터 가져오기 (소스 <https://bit.ly/wine-date>)

```
import pandas as pd  
wine = pd.read_csv('https://bit.ly/wine-date')
```

- head() 메서드로 처음 5개의 샘플을 확인

```
wine.head()
```



- alcohol : 알코올도수,
- sugar : 당도
- pH : pH
- class : 타깃값
- 0 이면 레드 와인, 1이면 화이트 와인
- 1 화이트와인이 양성 클래스

	alcohol	sugar	pH	class
0	9.4	1.9	3.51	0.0
1	9.8	2.6	3.20	0.0
2	9.8	2.3	3.26	0.0
3	9.8	1.9	3.16	0.0
4	9.4	1.9	3.51	0.0

로지스틱 회귀로 와인 분류하기



- `info()` 메서드: 데이터프레임의 각 열의 데이터 타입과 누락된 데이터가 있는지 확인

```
wine.info()
```

```
wine.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6497 entries, 0 to 6496
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   alcohol    6497 non-null   float64
1   sugar      6497 non-null   float64
2   pH         6497 non-null   float64
3   class      6497 non-null   float64
dtypes: float64(4)
memory usage: 203.2 KB
```

로지스틱 회귀로 와인 분류하기



- describe() 매사드: 열에 대한 간략한 통계(최소, 최대, 평균값 등)를 출력

```
wine.describe()
```

		alcohol	sugar	pH	class
	count	6497.000000	6497.000000	6497.000000	6497.000000
평균	mean	10.491801	5.443235	3.218501	0.753886
표준편차	std	1.192712	4.757804	0.160787	0.430779
최소	min	8.000000	0.600000	2.720000	0.000000
1사분위수	25%	9.500000	1.800000	3.110000	1.000000
중간값 / 2사분위수	50%	10.300000	3.000000	3.210000	1.000000
3사분위수	75%	11.300000	8.100000	3.320000	1.000000
최대	max	14.900000	65.800000	4.010000	1.000000

▲ 도수, 당도, pH 스케일이 다름

StandardScaler 클래스를 사용해 특성을 표준화 필요

로지스틱 회귀로 와인 분류하기



- 판다스 데이터프레임을 넘파이로 변환

```
data = wine[['alcohol', 'sugar', 'pH']].to_numpy()
target = wine['class']
```

- [노트]실습과 결괏값이 같도록 random_state를 42로 고정

```
from sklearn.model_selection import train_test_split
train_input, test_input, train_target, test_target = train_test_split(
    data, target, test_size=0.2, random_state=42)
```

- 훈련 세트와 테스트 세트의 크기 확인

```
print(train_input.shape, test_input.shape)
```

→ (5197, 3) (1300, 3)

로지스틱 회귀로 와인 분류하기



- StandardScaler 클래스를 사용해 훈련 세트 전처리와 테스트 세트 변환

```
from sklearn.preprocessing import StandardScaler  
ss = StandardScaler()  
ss.fit(train_input)  
train_scaled = ss.transform(train_input)  
test_scaled = ss.transform(test_input)
```


로지스틱 회귀로 와인 분류하기



- 표준점수로 변환된 train_scaled와 test_scaled를 사용해 로지스틱 회귀 모델을 훈련

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()
lr.fit(train_scaled, train_target)

print(lr.score(train_scaled, train_target))
0.7808350971714451
print(lr.score(test_scaled, test_target))
0.7776923076923077

print(lr.coef_, lr.intercept_)
[[ 0.51270274  1.6733911 -0.68767781]] [1.81777902]
```

▲ 훈련 세트와 테스트 세트의 점수가 모두 낮아 다소 과소적합

로지스틱 회귀로 와인 분류하기



설명하기 쉬운 모델과 어려운 모델

- 모델을 설명하기 위한 보고서 작성을 위해, 로지스틱 회귀가 학습한 계수와 절편을 출력

```
print(lr.coef_, lr.intercept_)
```

→ `[[ 0.51268071 1.67335441 -0.68775646]] [1.81773456]`

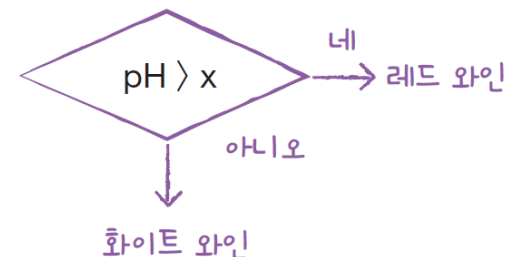
보고서

작성자 : 혼공머신

이 모델은 알코올 도수 값에 0.51268071를 곱하고, 당도에 1.67335441을 곱하고, pH 값에 -0.68775646을 곱한 다음 모두 더합니다. 마지막으로 1.81773456를 더합니다. 이 값이 0보다 크면 화이트 와인, 작으면 레드 와인입니다. 현재 약 78% 정도를 정확히 화이트 와인으로 분류했습니다. 모델의 출력 결과는...



순서도로 표현



결정 트리(Decision Tree)



- 사이킷런의 DecisionTreeClassifier 클래스를 사용해 결정 트리 모델을 훈련
 - fit() 메서드를 호출해서 모델을 훈련한 다음 score() 메서드로 정확도를 평가

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt = DecisionTreeClassifier(random_state=42)  
dt.fit(train_scaled, train_target)
```

```
print(dt.score(train_scaled, train_target))
```

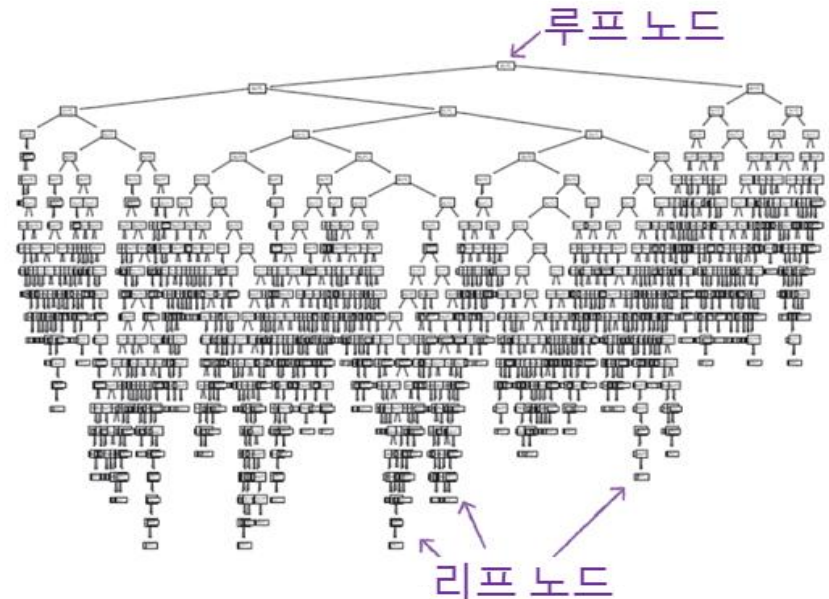
```
0.996921300750433
```

```
print(dt.score(test_scaled, test_target))
```

```
0.8592307692307692
```

```
import matplotlib.pyplot as plt  
from sklearn.tree import plot_tree
```

```
plt.figure(figsize=(10,7))  
plot_tree(dt)  
plt.show()
```

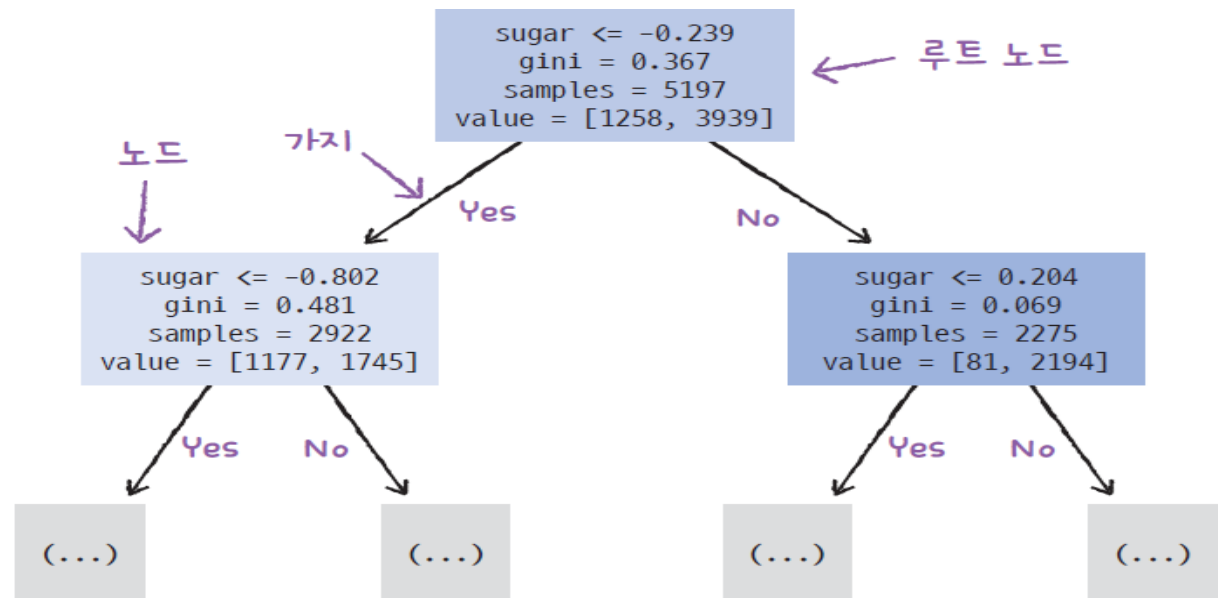


결정 트리



- 사이킷런의 **DecisionTreeClassifier** 클래스를 사용해 결정 트리 모델을 훈련
 - `max_depth` 매개변수를 1로 주면 루트 노드를 제외하고 하나의 노드를 더 확장
 - `filled` 매개변수에서 클래스에 맞게 노드의 색깔 조정
 - `feature_names` 매개변수에는 특성의 이름을 전달

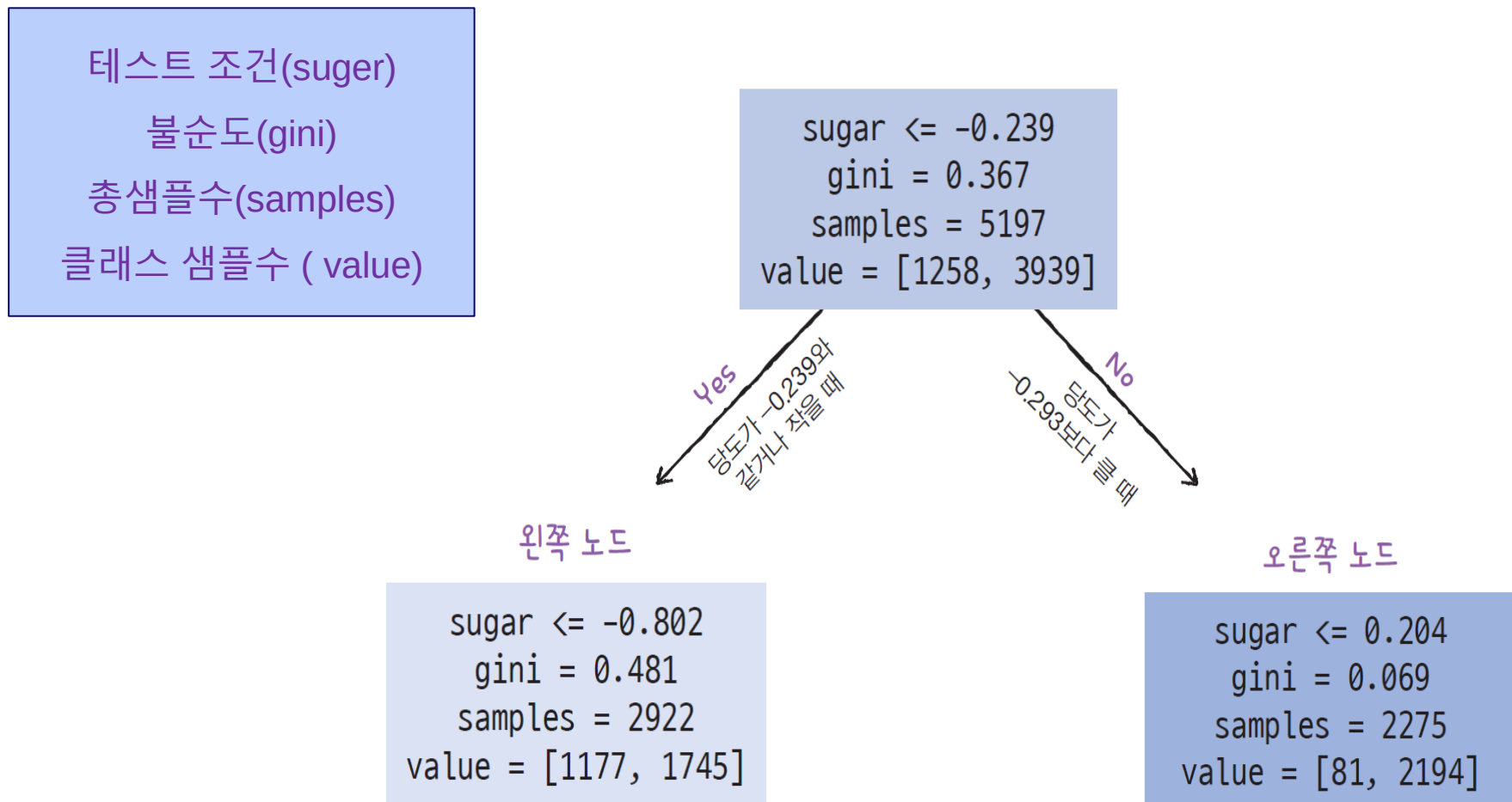
```
plt.figure(figsize=(10,7))  
plot_tree(dt, max_depth=1, filled=True,  
          feature_names=['alcohol', 'sugar', 'pH'])  
plt.show()
```





결정 트리

- 사이킷런의 DecisionTreeClassifier 클래스를 사용해 결정 트리 모델을 훈련





- 불순도(impurity)

- 한 노드 안에 클래스가 얼마나 섞여 있는지를 측정함

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

- p_k : 클래스 k 의 비율
- 값이 0이면 완전 순수(한 클래스만 존재)
- 값이 클수록 섞여 있음



■ 지니 불순도

- gini는 지니 불순도(Gini impurity)를 의미
- DecisionTreeClassifier 클래스의 criterion 매개변수의 기본값이 'gini'
- criterion 매개변수: 노드에서 데이터를 분할할 기준을 정함
- 앞의 그린 트리에서 루트 노드는 당도 -0.239를 기준으로 왼쪽과 오른쪽 노드로 나눌 때, criterion 매개변수에 지정한 지니 불순도를 사용

$$\text{지니 불순도} = 1 - (\text{음성 클래스 비율}^2 + \text{양성 클래스 비율}^2)$$

- 루트 노드는 총 5,197개의 샘플, 그중에 1,258개가 음성 클래스, 3,939개가 양성 클래스인 경우의 지니불순도

$$1 - ((1258 / 5197)^2 + (3939 / 5197)^2) = 0.367$$

- 순수 노드: 노드에 하나의 클래스만 있어, 지니 불순도가 0인 노드

$$1 - ((0 / 100)^2 + (100 / 100)^2) = 0$$



- 불순도(impurity)

- 정보 이득(information gain): 분할 전후의 불순도 감소량
(분할로 인해 불순도가 얼마나 감소했는가를 측정하는 값)

$$Information\ Gain = H(\text{부모}) - \sum_i \frac{n_i}{n} H(\text{자식}_i)$$

- 정보 이득 = 부모 불순도 - 자식 노드들의 가중 평균 불순도

⇒ 정보 이득이 크다 : 자식 노드 불순도가 많이 감소, 노드가 잘 분리, 좋은 분할

⇒ 정보 이득이 작다 : 분할해도 여전히 섞여 있음, 불순도 거의 감소 안함, 나쁜 분할



■ 가지치기

- 결정 트리에서 자라날 수 있는 트리의 최대 깊이를 지정
- DecisionTreeClassifier 클래스의 max_depth 매개변수를 3으로 지정하여 모델 만들기

```
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(train_scaled, train_target)
print(dt.score(train_scaled, train_target))
print(dt.score(test_scaled, test_target))
```

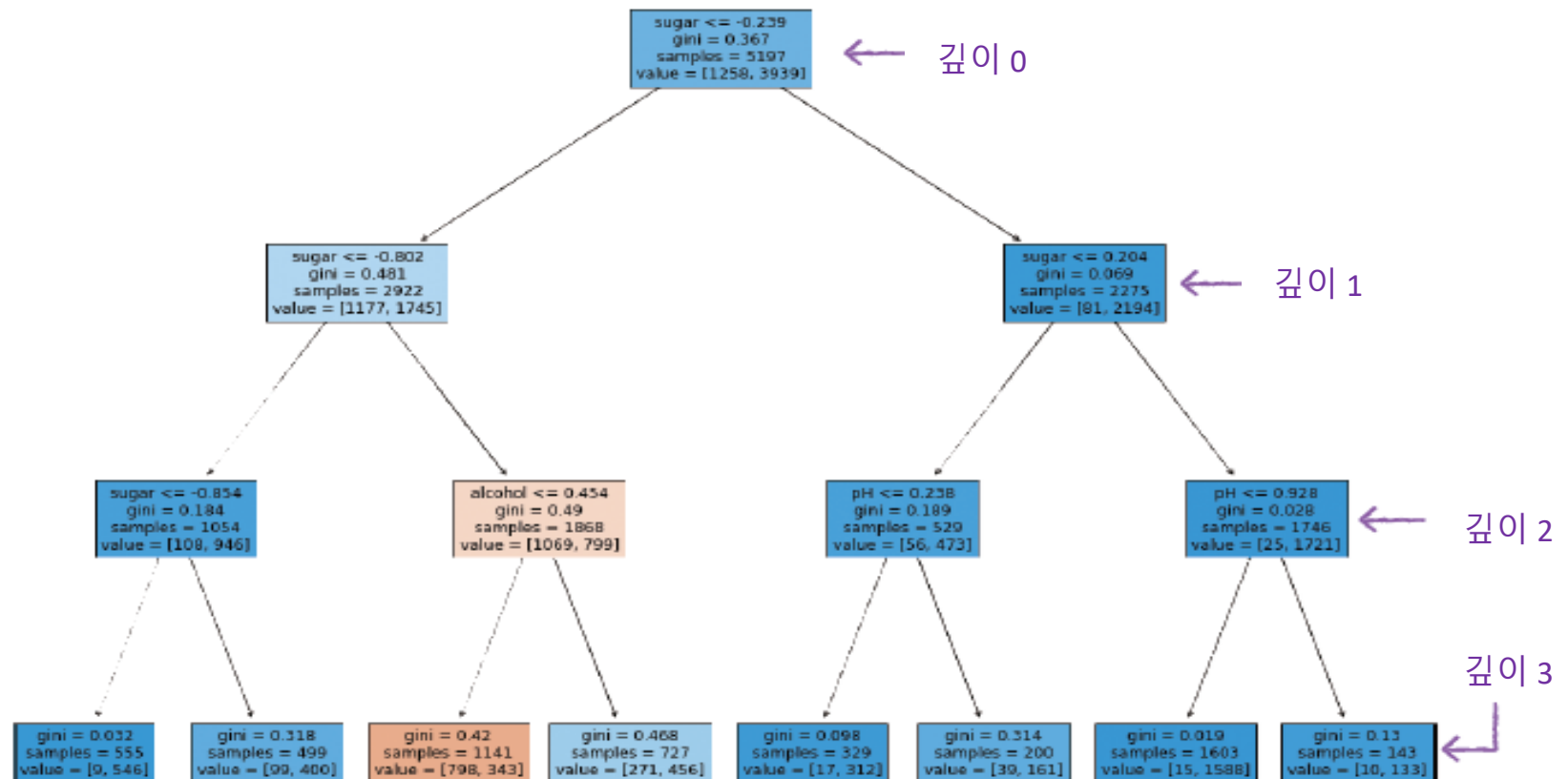
→ 0.8454877814123533
0.8415384615384616

결정 트리



가치치기

```
plt.figure(figsize=(20,15))  
plot_tree(dt, filled=True, feature_names=['alcohol', 'sugar', 'pH'])  
plt.show()
```



결정 트리 – 표준화 전처리 안함



■ 가지치기

- 특성값의 스케일은 결정 트리 알고리즘에 아무런 영향을 미치지 않으므로 표준화 전처리를 할 필요가 없음
- 전처리하기 전의 훈련 세트(train_input)와 테스트 세트(test_input)로 결정 트리 모델을 다시 훈련

```
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(train_input, train_target)
print(dt.score(train_input, train_target))
print(dt.score(test_input, test_target))
```

0.8454877814123533

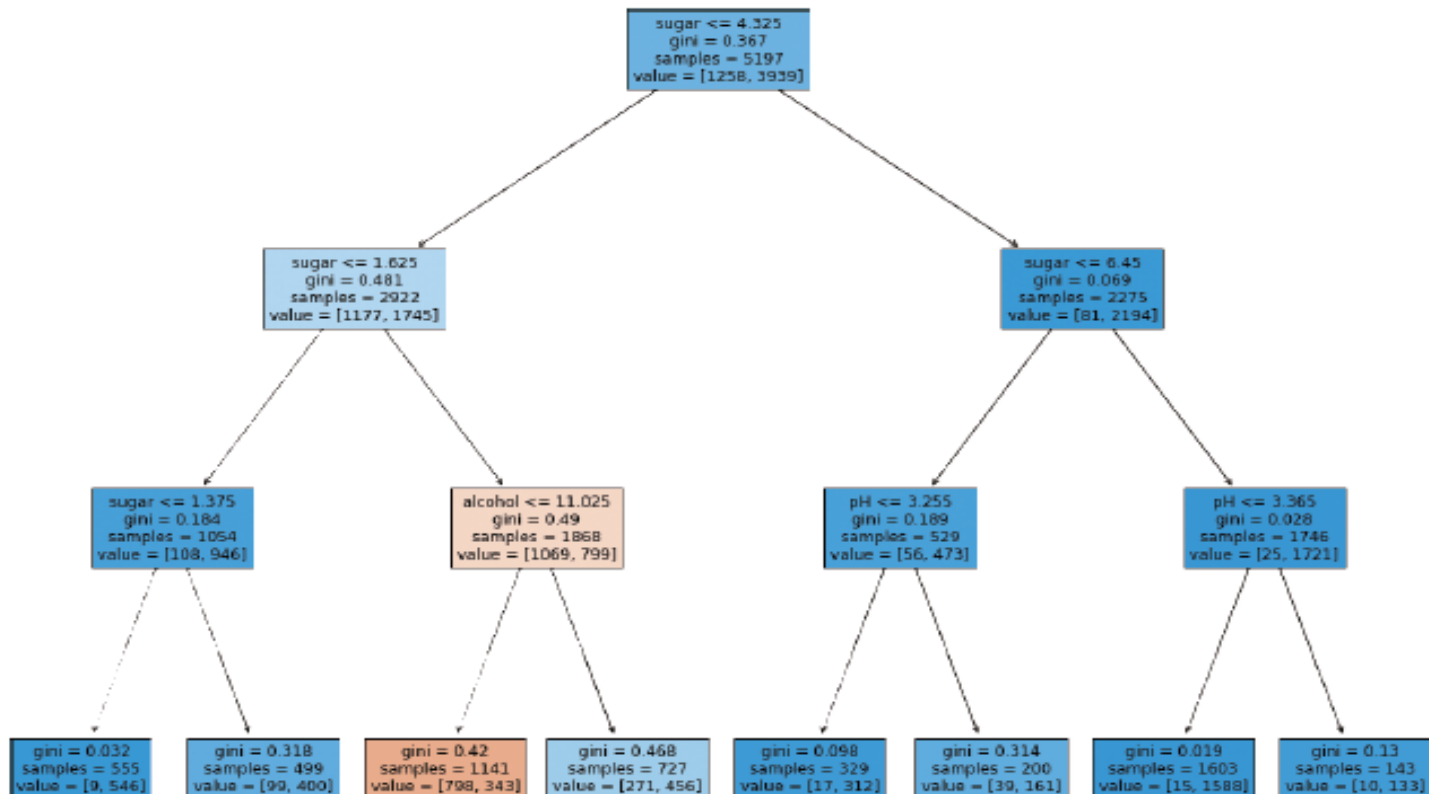
0.8415384615384616

결정 트리 — 표준화 전처리 안함



가치치기

```
plt.figure(figsize=(20,15))  
plot_tree(dt, filled=True, feature_names=['alcohol', 'sugar', 'pH'])  
plt.show()
```



결정 트리



▪ 특성 중요도

- 결정 트리는 어떤 특성이 가장 유용한지 나타내는 특성 중요도를 계산가능
- 앞의 트리에서는 루트 노드와 깊이 1에서 당도를 사용했기 때문에 아마도 당도(sugar)가 가장 유용한 특성 중 하나일 것임
- 특성 중요도는 결정 트리 모델의 `feature_importances_` 속성에 저장됨

```
print(dt.feature_importances_) → [0.12345626      0.86862934      0.0079144 ]
```

- 특성 중요도는 각 노드의 정보 이득과 전체 샘플에 대한 비율을 곱한 후 특성별로 더하여 계산
- 특성 중요도를 활용하면 결정 트리 모델을 특성 선택에 이용할 수 있음

이해하기 쉬운 결정 트리 모델 (문제해결 과정)



- 문제
 - 알코올 도수, 당도, pH 데이터를 기준으로 화이트 와인을 골라내는 이진 분류 로지스틱 회귀 모델을 훈련
 - 설명하고 이해하기 쉬운 방법의 모델이 필요
- 해결
 - 결정 트리를 사용해 레드 와인과 화이트 와인을 분류하는 문제 해결
 - 특성을 더 추가하지 않고도 결정 트리의 성능이 로지스틱 회귀 모델보다 더 뛰어남
 - 결정 트리는 깊이가 너무 깊지 않다면 비교적 설명하기 쉬움
 - 결정 트리가 어떻게 데이터를 분할하는지 이해하기 위해 불순도 개념과 정보 이득에 대해 학습

이해하기 쉬운 결정 트리 모델 (문제해결 과정)



- 해결
 - 결정 트리는 비교적 비전문가에게도 설명하기 쉬운 모델을 만들어줌
 - 결정 트리는 많은 앙상블 학습 알고리즘의 기반이며, 앙상블 학습은 신경망과 함께 가장 높은 성능의 인기 알고리즘



- 키워드로 끝나는 핵심 포인트

- 결정 트리는 예/아니오에 대한 질문을 이어나가면서 정답을 찾아 학습하는 알고리즘
 - 비교적 예측 과정을 이해하기 쉽고 성능도 우수
- 불순도는 결정 트리가 최적의 질문을 찾기 위한 기준
 - 사이킷런은 지니 불순도와 엔트로피 불순도를 제공
- 정보 이득은 부모 노드와 자식 노드의 불순도 차이
 - 결정 트리 알고리즘은 정보 이득이 최대화되도록 학습
- 결정 트리는 제한 없이 성장하면 훈련 세트에 과대적합되기 쉬움
 - 가지치기는 결정 트리의 성장을 제한하는 방법
 - 사이킷런의 결정 트리 알고리즘은 여러 가지 가지치기 매개변수를 제공
- 특성 중요도는 결정 트리에 사용된 특성이 불순도를 감소하는데 기여한 정도를 나타내는 값



- 핵심 패키지와 함수
 - pandas
 - `info()`: 데이터프레임의 요약된 정보를 출력
 - `describe()`: 데이터프레임 열의 통계 값을 제공
 - scikit-learn
 - `DecisionTreeClassifier`: 결정 트리 분류 클래스
 - `plot_tree()`: 결정 트리 모델을 시각화

확인 문제



1. 다음 중 결정트리(Decision Tree)의 특징으로 가장 적절한 것은?

- ① 데이터의 평균값을 계산하여 예측한다.
- ② 데이터를 여러 조건으로 분할하여 분류 또는 예측한다.
- ③ 모든 데이터를 하나의 군집으로 묶는다.
- ④ 데이터의 차원을 축소한다.

2. 다음 중 지니(Gini) 불순도에 대한 설명으로 가장 옳은 것은?

- ① 클래스 비율 중 가장 큰 값을 선택하여 계산한다.
- ② 클래스 비율과 클래스 비율의 로그값을 곱한 후 모두 더하여 계산한다.
- ③ 각 클래스 비율을 제곱하여 모두 더한 값을 1에서 빼서 계산한다.
- ④ 부모 노드의 깊이와 자식 노드의 깊이 차이로 계산한다.

확인 문제



3. 다음 중 지니 불순도가 가장 낮은 노드는?

- ① 클래스 비율 (0.5, 0.5)
- ② 클래스 비율 (0.6, 0.4)
- ③ 클래스 비율 (0.7, 0.3)
- ④ 클래스 비율 (1.0, 0.0)

4. 어떤 노드의 클래스 비율이 $A=0.8$, $B=0.2$ 일 때 지니 불순도는?

- ① 0.16
- ② 0.32
- ③ 0.64
- ④ 0.80



5. 결정트리 모델이 과적합(Overfitting)될 가능성이 가장 높은 경우는?

- ① 트리의 깊이를 너무 크게 설정한 경우
- ② 학습 데이터가 충분한 경우
- ③ 특성의 개수가 적은 경우
- ④ 노드 수를 제한한 경우

6. 결정트리에서 특성 중요도(Feature Importance)가 의미하는 것은?

- ① 특성의 데이터 타입
- ② 특성이 분할에 기여한 정도
- ③ 특성의 평균값
- ④ 특성의 개수



7. 결정트리에서 가지치기(Pruning)를 수행하는 주된 이유는?

- ① 학습 속도를 느리게 하기 위해
- ② 특성 수를 증가시키기 위해
- ③ 과적합을 줄이기 위해
- ④ 데이터를 정규화하기 위해

8. 다음 중 결정트리의 장점으로 가장 적절한 것은?

- ① 반드시 데이터 표준화가 필요하다.
- ② 결과를 시각화하고 해석하기 쉽다.
- ③ 범주형 데이터를 사용할 수 없다.
- ④ 결측치를 자동으로 제거한다.



9. 다음 중 결정트리의 깊이(max_depth)를 증가시킬 때 나타날 가능성이 가장 높은 현상은?

- ① 편향(Bias) 증가, 분산(Variance) 감소
- ② 편향 감소, 분산 증가
- ③ 편향 증가, 분산 증가
- ④ 편향 감소, 분산 감소

10. 결정트리 모델을 학습한 후 각 특성의 중요도를 확인하기 위해 사용하는 속성은 무엇인가?

- ① feature_importance
- ② feature_importances
- ③ feature_importances_
- ④ importance_feature_

실습 문제



붓꽃(Iris) 데이터로 결정트리를 학습한 후 다음을 수행하시오.

- ① 학습/테스트 데이터 분리
- ② 결정트리 생성 ($\text{max_depth}=3$)
- ③ 모델 학습
- ④ 정확도 출력
- ⑤ 특성 중요도 출력
- ⑥ 결정트리 시각화

Q & A

